

激光器控制系统上位机说明

本项目是基于 Qt Widgets 和 QtSerialPort 的三路激光器控制上位机。普通操作员页面和开发者页面共用 `LaserController`，所有串口发送、顺序联锁、缓升缓降、配置读取都集中在控制核心中，界面层只发起请求。

目录

- [1. 控制核心](#)
- [2. 串口协议](#)
- [3. 顺序联锁](#)
- [4. 统一启动曲线](#)
- [5. 开发者 TRY](#)
- [6. 普通操作员页面](#)
- [7. 配置文件](#)
- [8. 温度检测与就绪状态](#)
- [9. 全局发送限速 \(15Hz\)](#)
- [10. 安全注意](#)
- [11. 重启恢复策略](#)
- [附录：更新记录](#)

1. 控制核心

当前手写源码职责：

- `main.cpp` 负责 Qt 高 DPI 设置、UTF-8 locale、全局 `Microsoft YaHei` 字体，并启动普通操作员窗口 `operatorForm`。
- `operatorForm` 是程序首屏，提供 L1 种子、L2 预放、L3 百分比功率和串口连接入口；开发者入口需要密码验证，进入后隐藏普通页面。
- `widget` 是开发者页面，复用同一个 `LaserController`，提供串口日志、三路手动步进/目标值输入、参数保存、温度旁路和 TRY 扫描。
- `LaserChart` 同时绘制设定值曲线和实测值曲线；`PillSpinBox` 是普通页面的百分比功率输入控件，左右区域自绘 `- / +` 和 `%`。
- `LaserController` 是唯一控制核心，普通页面和开发者页面都只向它提交请求，不各自维护串口或联锁状态。

控制链路：

```
普通页面 / 开发者页面 / TRY
-> LaserController 联锁判断
-> 定时器分步缓升或缓降
-> sendLaserCommand()
-> STM32 v1.3 文本帧串口指令 <CMD=X,CHK=YY>
-> 解析 STM32 状态回包，更新上位机 setpoint、ready、实测值和曲线
```

`LaserController` 会在三个层面兜底：

- 请求入口：`canAdjustLaser()` 判断当前方向是否允许。
- 目标调节：`setLaserTarget()` 启动定时器前再次判断目标方向。
- 串口发送：`sendLaserCommand()` 每次发文本帧前再次检查联锁。

因此即使 UI 状态刷新不及时，也不会绕过内部顺序联锁。

2. 串口协议

上位机已适配 STM32 v1.3 文本帧协议。命令字符仍沿用下表的 0~9，但真实串口不再发送裸单字节，而是统一封装为：

```
<CMD=X,CHK=YY>
```

- `CMD` 为单字符命令。
- `CHK` 为 `CMD` 字符 ASCII 值的低 8 位，两位十六进制表示，例如 `CMD=1` 对应 `CHK=31`，`CMD=D` 对应 `CHK=44`。
- 查询命令：`A` 查询 L1，`B` 查询 L2，`C` 查询 L3，`D` 查询全部状态。
- 每次真实串口连接或自动重连成功后，上位机会延迟 500 ms 发送一次 `D` 查询；若被全局限速挡住，会每 200 ms 最多重试 5 次；空闲状态下也会周期性查询全部状态，避免软件异常退出后重新打开时仍按初始状态显示。
- 真实串口连接后默认进入“状态同步中”，在收到 L1/L2/L3 三路 `[LaserX] ... SET_I=...` 状态行并更新本地状态前，所有激光器控制入口保持锁定。

典型帧：

```
<CMD=1,CHK=31>
<CMD=D,CHK=44>
```

指令	作用	DAC 增量	大约电流增量
'0'	L1 粗调降	-31	-10 mA
'1'	L1 粗调升	+31	+10 mA
'2'	L1 细调降	-3	-1 mA
'3'	L1 细调升	+3	+1 mA
'4'	L2 粗调升	+31	+10 mA
'5'	L2 粗调降	-31	-10 mA
'6'	L2 细调升	+3	+1 mA
'7'	L2 细调降	-3	-1 mA
'8'	L3 升	+20	+100 mA
'9'	L3 降	-20	-100 mA

L1/L2 的可选步长只能是 1 mA 或 10 mA。L3 由硬件协议固定为 100 mA。

STM32 v1.3 状态回包示例：

```
[Laser1] READY=YES ENABLE=ON DAC=304 DAC_V=0.245V SET_I=0.098A ADC_RAW=0
ADC_V=0.000V FB_I=0.000A
[Laser2] READY=YES ENABLE=ON DAC=1427 DAC_V=1.150V SET_I=0.460A ADC_RAW=0
ADC_V=0.000V FB_I=0.000A
[Laser3] READY=YES ENABLE=ON DAC=838 DAC_V=0.675V SET_I=5.000A ADCV_RAW=0
ADCV=0.000V FB_V=2.000V ADCI_RAW=0 ADCI=0.000V FB_I=0.800A
```

上位机解析规则：

- `READY=YES/NO` 更新 `laserXRawReady`，再由 `updateLaserDependencies()` 计算最终 ready 联锁。
- `SET_I=...A` 校正上位机 `currentLaserMa()`，避免仅凭 `serialPort->write()` 成功就推进 setpoint。
- `FB_I=...A` 更新实测电流曲线。
- 控制命令发出后，真实串口模式会等待对应通道状态回包；收到 `SET_I` 后清除 pending，ramp 才继续发送下一步。
- 收到 `ERROR` 时清除 pending；如果 ramp 正在执行，则中止 ramp。

3. 顺序联锁

开机顺序： `L1 -> L2 -> L3`

目标	允许条件
L1 升高	串口可用，L1 就绪，且 L2=0、L3=800
L2 升高	L1/L2 就绪，L1 达到 L1EnableL2Ma，且 L3=800
L3 升高	L1/L2/L3 就绪，L1 和 L2 都达到启动阈值

关机顺序： `L3 -> L2 -> L1`

目标	允许条件
L3 降低	允许先降 L3
L2 降低	必须先保证 L3 回到 L3SafeOffMa，默认 800 mA
L1 降低	必须先保证 L2 回到 0 mA，且 L3 回到 800 mA

4. 统一启动曲线

L1 和 L2 的启动曲线配置：

- `[StartupL1]`：L1 普通启动和开发者 TRY 的 L1 前段共用（三段式曲线）。
- `[StartupL2]`：L2 单段缓升，从 0 直接升至目标电流。

L1 三段式曲线参数：

参数	含义
HighMa	第一段上升目标
MiddleMa	第二段回落目标
FinalMa	第三段最终工作电流
RiseDurationMs	当前值到高点的目标时长
FallMiddleDurationMs	高点到中间点的目标时长
FallFinalDurationMs	中间点到最终工作电流的目标时长
StepMa	启动步长，只能是 1 或 10

L2 单段缓升参数：

参数	含义
FinalMa	目标工作电流
RiseDurationMs	缓升总时长
StepMa	启动步长，只能是 1 或 10

默认值：

通道	曲线	步长
L1	0 -> 850 -> 200 -> 98 mA	1 mA
L2	0 -> 460 mA (单段)	10 mA

普通操作员页面点击 L1 或 L2 开启时，L1 走三段曲线，L2 走单段缓升。开发者 TRY 的 L1 前段走同一套 L1 曲线，L2 也直接复用 [StartupL2] 的目标、步长和时长，不再维护第二套 L2 TRY 参数。

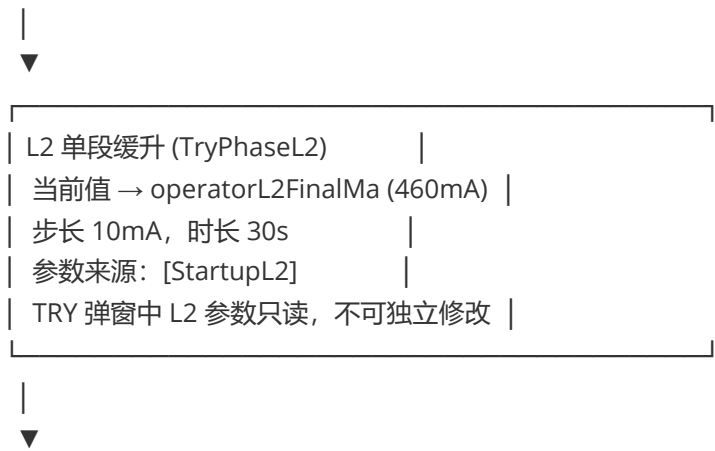
5. 开发者 TRY

5.1 状态机

阶段	动作	方向	终点	下一步
TryPhase1	L1 升	↑	850mA	→ TryPhase2
TryPhase2	L1 降	↓	200mA	→ TryPhase3
TryPhase3	L1 降	↓	98mA	→ TryPhaseL2
TryPhaseL2	L2 升	↑	460mA	→ TryPhaseL3
TryPhaseL3	L3 升	↑	5000mA	→ Done

5.2 扫描流程

L1 启动曲线完成 (98mA)



L3 扫描 (TryPhaseL3)

800 → 5000 mA

TRY 执行顺序:

L1 StartupL1 三段启动

- > L2 单段缓升到 StartupL2/FinalMa
- > L3 扫描到 DeveloperTry/L3TargetMa

5.3 L2 参数统一化

L2 在普通页面开启和 TRY 扫描中**共用同一套参数**:

- **参数唯一来源**: [StartupL2] 配置段 (FinalMa、RiseDurationMs、StepMa)
- **L2 参数弹窗** (on_laser2ParamsButton_clicked): 只保留 3 个字段 —— 最终工作电流、缓升时长、启动步长。保存时自动将 TRY 参数同步为相同值。
- **TRY 扫描弹窗** (on_tryButton_clicked): L2 目标/步长/时间显示为只读, 标注"来自 L2 参数", 不可独立修改。
- loadConfig(): 启动时强制 tryL2TargetMa = startupL2FinalMa, 避免旧 ini 残留的 DeveloperTry/L2* 值造成分叉。
- saveDeveloperLaserParameters(): DeveloperTry/L2* 仍写入, 但数值从 StartupL2 同步, 仅作为兼容镜像。

5.4 注意事项

- L2 在 TRY 中只有一段缓升, 不使用三段式曲线。
- TRY 每一步仍然调用 LaserController::setLaserTarget(), 不会绕过联锁。
- **TRY 扫描期间, 开发者页面的 +/- 按钮和 SpinBox 会被自动锁定**, 避免手动命令插入自动扫描流程。锁定会持续到扫描完成或手动停止, 不会在每段 ramp 间隙短暂解锁, 杜绝按钮闪烁。
- TRY 的 L1 阶段复用 [StartupL1] 的三段曲线参数; L3 的扫描目标/步长/时长仍在 TRY 弹窗中独立设置, 保存在 [DeveloperTry]。

6. 普通操作员页面

程序启动后默认显示普通操作员页面。该页面面向日常操作，只暴露 L1、L2 开关、L3 功率百分比和串口连接；开发者页面从右上角开发者入口打开，密码校验成功后复用同一个 `LaserController`，普通页面会临时隐藏，开发者窗口关闭后再恢复。

普通操作员页面提供串口连接入口：

- 串口下拉框来自 `LaserController::availablePortNames()`，用于选择当前要连接的端口。
- 连接/断开 按钮调用同一个 `LaserController::openSerial()` / `closeSerial()`，不会创建第二套串口控制链路。
- 连接状态会显示在普通页面底部；串口断开、热插拔或自动重连时，普通页和开发者页会同步刷新。
- 未连接时，L1、L2 和 L3 功率输入都会被内部联锁锁住，提示“请先连接串口”。
- L3 功率输入使用 `PillSpinBox`，百分比范围来自 `[L3OperatorPower]`；`operatorPowerPercentStep()` 会保证一次 `+/-` 至少跨过一个 L3 硬件电流档位，避免点击后又被实时回显折回原百分比。
- L3 普通功率回显不再直接按线性公式把实际 mA 反算成百分比，而是由控制核心内建百分比档位表：每个百分比先映射到一个按硬件步长对齐后的中心电流，实际 setpoint 落在该中心值约 ± 30 mA 内时就视为命中该百分比；普通页面会记住最近一次用户请求的百分比，L3 调整中或进入容差范围后继续显示该目标百分比，避免硬件单步浮动导致百分比在相邻档之间反复横跳。

L3 百分比 → 电流档位对照表

L3 范围 800~5000 mA，硬件步长 100 mA，共 43 个档位。99 个百分比点（2%~100%）按 `operatorPowerCenterMaForPercent()` 线性插值后对齐到最近的 100 mA 整数倍，相邻百分比可能映射到同一电流中心。

百分比	中心电流		百分比	中心电流		百分比	中心电流
2%~3%	800 mA		36%~38%	2300 mA		71%~73%	3800 mA
4%~5%	900 mA		39%~40%	2400 mA		74%~75%	3900 mA
6%~7%	1000 mA		41%~42%	2500 mA		76%~77%	4000 mA
8%~10%	1100 mA		43%~45%	2600 mA		78%~80%	4100 mA
11%~12%	1200 mA		46%~47%	2700 mA		81%~82%	4200 mA
13%~14%	1300 mA		48%~49%	2800 mA		83%~84%	4300 mA
15%~17%	1400 mA		50%~52%	2900 mA		85%~87%	4400 mA
18%~19%	1500 mA		53%~54%	3000 mA		88%~89%	4500 mA
20%~21%	1600 mA		55%~56%	3100 mA		90%~91%	4600 mA
22%~24%	1700 mA		57%~59%	3200 mA		92%~94%	4700 mA
25%~26%	1800 mA		60%~61%	3300 mA		95%~96%	4800 mA
27%~28%	1900 mA		62%~63%	3400 mA		97%~98%	4900 mA
29%~31%	2000 mA		64%~66%	3500 mA		99%~100%	5000 mA

百分比	中心电流		百分比	中心电流		百分比	中心电流
32%~33%	2100 mA		67%~68%	3600 mA			
34%~35%	2200 mA		69%~70%	3700 mA			

每个档位的容差窗口为 中心 ± 30 mA，相邻窗口间距至少 40 mA，无重叠。回显时 `operatorPowerMaToPercent()` 扫描上表选择距离实际电流最近的百分比；`requestedPowerPercent` 记忆确保调整期间优先保持用户设置的目标百分比。

启动顺序：

步骤	操作	系统行为
1	在普通页面选择串口并点击 连接	未连接时所有操作锁住
2	点击 L1 开/关（种子）	L1 按 <code>StartupL1</code> 三段启动，完成后按钮才显示已开启
3	点击 预放开/关	L2 按 <code>StartupL2</code> 单段缓升，完成后按钮才显示已开启
4	调整功率百分比	2%~100% 映射到 L3 800~5000 mA，并按 100 mA 对齐；实际回包允许约 ± 30 mA 档位容差

关机顺序：

步骤	操作	系统行为
1	将功率百分比调回 2%	L3 回到 800 mA
2	关闭预放 L2	只有 L3=800 时才允许关闭
3	关闭 L1	只有 L2=0 且 L3=800 时才允许关闭

按钮显示规则：

- 未满足顺序条件时按钮置灰或提交后提示原因。
- 正在开启/关闭时显示“正在开启...”或“正在关闭...”。
- 只有控制核心发出完成信号后，按钮才切换为“已开启/已关闭”。

温度旁路安全提示：

- 该提示区域也用于显示下位机状态查询进度。真实串口连接后，如果尚未收到三路状态回包，会优先显示：“正在查询下位机状态，完成前禁止操作激光器。”
- 只有下位机状态同步完成后，普通页面才根据温度旁路开关决定是否显示温度旁路警告。
- 当开发者开启温度旁路后，普通页面会在面板下方显示醒目的警告条：“警告：温度就绪旁路已开启。上位机将忽略下位机温度就绪状态，仅保留顺序联锁；最终温度保护依赖下位机。”
- 该警告条持续显示，直至开发者关闭旁路后才自动隐藏，避免操作员误以为上位机已验证温度就绪。
- 警告条位于功率调节区域下方、串口连接栏上方，不会遮挡主要操作控件。

7. 配置文件

程序启动时读取运行目录下的 `laser_config.ini`。如果文件不存在，控制核心会生成默认配置。

主要配置段：

配置段	作用
[Interlock]	L1/L2 启动阈值和安全关闭态
[Range]	L1/L2/L3 设定值上限
[StartupL1]	L1 普通启动和 TRY 前段
[StartupL2]	L2 普通启动和 TRY L2 段，唯一 L2 启动/扫描参数源
[Step]	与 STM32 指令对应的固定步长
[L3operatorPower]	普通页面功率百分比到 L3 mA 的映射；运行时会按百分比档位中心和约 ±30 mA 容差进行回显匹配
[Ramp]	缓升间隔配置（默认/最小/手动均为 67ms，统一锁 15Hz）
[Temperature]	临时温度 ready 旁路开关，默认关闭
[DeveloperTry]	TRY L3 扫描目标和时长；L2 的 DeveloperTry 键仅作兼容镜像，实际以 [StartupL2] 为准

开发者页面每路都有独立参数按钮。保存时会写回对应配置段，并生成 `laser_config.ini.bak`。

8. 温度检测与就绪状态

当前系统有两层就绪状态：

laserXRawReady -> laserXReady

状态	含义	主要来源
laserXRawReady	STM32 原始温度/就绪状态	串口文本解析；不再由发送成功辅助置 true
laserXReady	上位机按依赖链计算后的最终就绪状态	updateLaserDependencies()

依赖链计算规则：

L1 ready = L1 rawReady
L2 ready = L1 ready && L2 rawReady
L3 ready = L1 ready && L2 ready && L3 rawReady

8.1 上位机依赖的温度/就绪输入

`serialHelper` 侧不直接读取温度 GPIO，只解析 STM32 通过串口输出的温度/状态文本。当前控制核心认可两类输入：

输入类型	当前代码支持的格式	作用
异步温度事件	<code>Laser1 temp state -> READY/NOT_READY</code> 、 <code>Laser2 temp state -> READY/NOT_READY</code>	更新 L1/L2 的 <code>rawReady</code>
状态查询回包	<code>[LaserX] READY=YES/NO ... SET_I=...A</code> <code>FB_I=...A</code>	更新三路 <code>rawReady</code> 、 setpoint 和实测电流

L3 的异步温度事件当前没有单独解析路径；上位机主要通过 `[Laser3] READY=YES/NO` 状态查询回包更新 L3 ready。最终硬件保护仍依赖 STM32 固件执行控制命令前的本地安全判断，上位机侧只作为顺序联锁和操作入口保护。

8.2 上位机解析路径

上位机在 `LaserController::serialPortReadyRead()` 中按行解析 STM32 输出。

当前支持的典型文本：

通道	就绪文本	未就绪文本
L1	<code>Laser1 temp state -> READY</code> 、 <code>[Laser1] READY=YES ...</code>	<code>Laser1 temp state -> NOT_READY</code> 、 <code>[Laser1] READY=NO ...</code>
L2	<code>Laser2 temp state -> READY</code> 、 <code>[Laser2] READY=YES ...</code>	<code>Laser2 temp state -> NOT_READY</code> 、 <code>[Laser2] READY=NO ...</code>
L3	<code>[Laser3] READY=YES ...</code>	<code>[Laser3] READY=NO ...</code>

解析后如果 `rawReady` 发生变化，会调用 `updateLaserDependencies()` 重新计算三路最终 ready，并通过 `readyChanged`、`stateChanged` 通知界面。

8.3 联锁使用的是 rawReady

安全联锁使用 `laserReadyForStartup()`，真实模式下读取的是 `laserRawReady()`，不是 `laserReady()`。

操作	温度/就绪条件
L1 升高	L1 rawReady
L2 升高	L1 rawReady && L2 rawReady
L3 升高	L1 rawReady && L2 rawReady && L3 rawReady
L3 降低	不要求温度 ready
L2 降低	不要求温度 ready，但要求 L3 已回安全态

操作	温度/就绪条件
L1 降低	不要求温度 ready，但要求 L2/L3 已回安全态

开发者页面会订阅 `readyChanged` 并显示状态灯；普通操作员页面不直接显示温度状态，而是通过按钮禁用和 tooltip 间接提示，例如“等待 L1 温度就绪”。

8.4 临时温度旁路模式

如果下位机暂时不能主动上传温度 ready，可以由开发者开启临时旁路：

```
[Temperature]
BypassReadyCheck=false
```

值	行为
false	默认安全行为：上位机必须收到 STM32 温度 ready 才允许升高
true	临时旁路：上位机忽略 rawReady，只保留顺序联锁和关机顺序

旁路开启后：

- `laserReadyForStartup()` 返回 true，不再因为缺少 rawReady 阻止升高。
- `canAdjustLaser()` 中的电流顺序仍然有效：开机仍是 L1 -> L2 -> L3，关机仍是 L3 -> L2 -> L1。
- **普通操作员页面**：面板下方会显示黄色警告条，提示“上位机未验证温度就绪，最终保护依赖下位机”。警告条持续显示，直到旁路关闭后才自动隐藏。
- **开发者页面**：底部“温度旁路”按钮变为橙色高亮状态（文字变为“温度旁路: 开启”），同时在日志区和串口栏之间显示独立的警告标签。开启前必须确认风险弹窗。
- 该模式只用于下位机暂不上传温度 ready 的临时场景，不等同于安全闭环。

8.5 当前状态与仍有限制

1. `updateLaserStatusFromSend()` 已不再把发送成功视为 ready。
rawReady 只来自 STM32 异步温度文本和状态查询回包中的 `READY=YES/NO`，避免 `serialPort->write()` 成功导致 false ready。
2. setpoint 已改为回包校正。
真实串口模式下，控制命令写入成功后不会立即推进 `currentLaserMa()`；上位机会等待 `[LaserX] ... SET_I=...A`，用 STM32 回包校正本地 setpoint。
3. 当前 pending 机制是轻量确认，不是完整 ACK/NAK。
`commandAckPending[3]` 会阻止同一通道在未收到状态回包前继续发送下一步 ramp 命令；收到 `ERROR`：会清除 pending 并中止 ramp。但它还没有命令序号、超时重试和 CRC。
4. `checkLaserStatus()` 已具备空闲查询能力。
串口连接成功后会发送一次 `D` 查询；空闲且全局发送间隔满足条件时，每 3 秒发送一次 `D` 查询。
ramp/TRY 忙碌期间不会插入查询，避免抢占控制命令。

5. 文本协议解析仍依赖固定输出格式。

当前解析目标是 STM32 v1.3 的 `LaserN temp state -> READY/NOT_READY` 和 `[LaserN] READY=... SET_I=... FB_I=...`。如果 STM32 输出字段名或格式变化，需要同步修改上位机解析。

6. L3 温度状态当前主要依赖状态查询回包。

上位机仍保留 L3 `rawReady/ready` 逻辑，但没有解析独立的 `Laser3 temp state -> ...` 异步文本。真实系统中 L3 是否需要独立温度事件，需要按 STM32 输出格式同步扩展解析。

8.6 建议改进方向

推荐后续逐步改成结构化闭环：

STM32 周期性上报 STAT

- > 上位机按时间戳更新 `rawReady`
- > 超时未收到则置为 `unknown/not ready`
- > 上位机发送 `CMD` 带序号和 `CRC`
- > STM32 回复 `ACK/NAK`，携带命令序号和执行后 `setpoint`
- > 上位机按序号确认后更新 `setpoint`

建议优先级：

1. STM32 周期性上报 L1/L2/L3 温度状态、enable 状态、`SET_I` 和 `FB_I`。
2. 上位机为 `rawReady` 和 `command pending` 增加超时保护，超时后禁止继续升高并提示通信异常。
3. 增加 `ACK/NAK`、命令序号、`CRC` 和超时重试。
4. 将文本协议升级为结构化协议，例如 `STAT,L1,READY,SET=98,FB=97.6`、`ACK,1,98`、`NAK,2,TEMP_NOT_READY`。

9. 全局发送限速（15Hz）

控制命令最终经过 `sendLaserCommand()`，状态查询经过 `sendStatusQuery()`。真实串口模式下两条路径共享 `globalSendTimer`，任意两条真实串口帧之间至少间隔 67ms；手动单步、ramp、TRY 和 A/B/C/D 状态查询都会受这个全局硬限速约束。

9.1 三层限速

界面请求（+/-按钮 / SpinBox / TRY / 操作员页面）

```
|
├─ 手动单步: adjustLaser()
|   └─ sendLaserCommand()
|       └─ 同通道 pending 回包时暂不继续发送
|       └─ 手动单步检查 lastSentTimers < minManualSendIntervalMs (67ms)
|       └─ 所有真实控制帧检查 globalSendTimer < 67ms
|
├─ 缓升/缓降: setLaserTarget() → processRampStep()
|   └─ rampIntervalForTarget()
|       └─ 有 durationMs: qMax(minRampIntervalMs (67ms), durationMs /
steps)
|       └─ 无 durationMs: defaultRampIntervalMs (67ms)
|
└─ TRY 扫描: tryStep()
    └─ tryIntervalForSegment() → qMax(kTryMinIntervalMs (67ms), totalTime /
steps)
```

状态查询 (A/B/C/D)

└─ `sendStatusQuery()`

└─ 同样检查 `globalSendTimer < 67ms`, 避免查询帧抢占控制帧节奏

常量	位置	值	作用
<code>kMinGlobalSendIntervalMs</code>	<code>lasercontroller.cpp</code>	67ms	真实串口全局硬限速, 控制帧和查询帧共享
<code>minManualSendIntervalMs</code>	<code>LaserConfig / [Ramp]</code>	67ms	手动单步的同通道最小间隔
<code>minRampIntervalMs</code>	<code>LaserConfig / [Ramp]</code>	67ms	<code>rampIntervalForTarget()</code> 指定时长时的下限
<code>defaultRampIntervalMs</code>	<code>LaserConfig / [Ramp]</code>	67ms	ramp 无指定时长时的默认间隔
<code>kTryMinIntervalMs</code>	<code>widget.cpp</code>	67ms	<code>tryIntervalForSegment()</code> 计算 TRY 步进间隔的下限

9.2 设计要点

- **全局硬地板**: `globalSendTimer` 是最后兜底, 保证真实串口任意两帧之间不会低于 67ms (约 15Hz), 包括跨通道控制和状态查询。
- **同通道 pending**: 真实串口模式下 `commandAckPending[3]` 会等待对应 `[LaserX] ... SET_I=...` 回包, 收到后才允许同通道 ramp 继续发下一步。
- **三值同步**: `defaultRampIntervalMs = minRampIntervalMs = minManualSendIntervalMs = 67ms`, 保证 ramp 和手动入口自身也不会主动要求超过 15Hz 的节奏。
- **TRY 的 `kTryMinIntervalMs`**: 即使配置的总时长很短导致反推间隔过小 (如 5 秒扫描 500 步 = 10ms), 也会被强制拉到 67ms。
- **Debug 模式本地模拟**: `DEBUG_MODE` 下不打开真实串口, `sendLaserCommand()` 会记录模拟文本帧并本地推进 setpoint, 便于调试 UI 流程。

10. 安全注意

- 生产环境应关闭 `DEBUG_MODE`, 否则不会真正通过串口控制硬件。
- 当前软件电流主要是上位机 setpoint, 实测电流只用于显示和曲线观察。
- 当前温度 ready 主要来自 STM32 事件文本和上位机本地状态, 尚未形成带超时的闭环心跳。
- `Temperature/BypassReadyCheck=true` 仅为临时旁路, 不代表温度安全已由上位机验证。
- 如果 setpoint 和实测值差异明显, 应暂停操作并检查硬件响应、温度就绪、驱动饱和或保护状态。
- 后续建议继续完善 ACK/NAK、序号、CRC、超时重试和实测电流闭环联锁。

11. 重启恢复策略

11.1 问题描述

上位机意外关闭（崩溃、断电、误关窗口）后重新启动，面临以下状态丢失：

丢失项	默认值	风险
<code>currentLaserMa()</code> (setpoint)	L1=0, L2=0, L3=800	显示值与真实输出脱节
<code>measuredLaserMa()</code>	-1（无效）	无法显示实测电流
<code>laserXRawReady</code>	false	所有通道被视为未就绪
<code>laserXReady</code>	false	联锁链全部断开

核心风险：如果关闭前 L3 实际在 5000mA，重启后上位机 setpoint 显示 800mA。此时操作员按 `+` 键：

- 上位机计算 $800 + 100 = 900$ ，发送 `'8'` 指令
- 下位机实际从 5000 $\rightarrow$ 5100mA
- **显示值与真实值完全脱节，持续操作会导致不可预期行为**

11.2 下位机现状与限制

仅从 `serialHelper` 当前代码看，上位机不能假设下位机会主动周期性完整上报全部状态；因此真实串口连接后必须主动发送 `D` 查询来恢复本地状态：

信息类型	上报方式	触发条件
温度就绪/ 未就绪	异步温度文本或状态行中的 <code>READY=YES/NO</code>	上位机只按收到的行更新 rawReady
输出设定值	状态行中的 <code>SET_I=...A</code>	用于校正 <code>currentLaserMa()</code> ，并清除对应通道 pending
实测电流	状态行中的 <code>FB_I=...A</code>	用于更新 <code>measuredLaserMa()</code> 和曲线
完整状态恢复	<code>D</code> 查询触发的三路 <code>[LaserX]</code> ... 回包	三路都收到后才开放控制权限

如果连接后没有拿到三路状态行，上位机会保持 `lowerDeviceStatesSynchronized=false`，普通页、开发者页、TRY 和 ramp 都不能发送控制命令。

11.3 方案分析

针对重启后状态恢复，分析了三种方案：

方案 A：降低 L3 电流探测状态（已否决）

在重连后自动发送一次 L3 降低命令 '9'，利用 STM32 执行后的电流回显来推断当前状态。

否决原因：

- **破坏性操作：**L3 降低是控制命令而非查询命令，会真实改变激光输出（如 5000 → 4900mA），属于带副作用的状态恢复。
- **只能拿到 L3 信息：**无法恢复 L1/L2 的 setpoint、ready、enable、联锁状态。
- **下位机可能拒执行：**若 L1/L2 温度未就绪，STM32 只会打印错误，不会输出 L3 电流。
- **上位机逻辑冲突：**重启后 setpoint 默认 800mA，普通降低路径可能因已达下限而判断为无需操作，需要额外绕开。

结论：不满足非破坏性恢复的安全设计要求。

方案 B：下位机新增专用查询命令（长期正式方案）

在上位机与 STM32 之间新增一条查询命令，例如上位机发送 '?'，STM32 立即回传完整状态：

```
STATE L1 set=98 meas=97.6 ready=1 enabled=1
STATE L2 set=460 meas=459.2 ready=1 enabled=1
STATE L3 set=5000 meas=4998.0 ready=1 enabled=1
```

上位机串口连接成功后发送一次查询，即可同步全部 currentLaserMa、measuredLaserMa、rawReady、联锁使能状态。

优点：无副作用、瞬时恢复、完整状态覆盖。

限制：需要修改 STM32 固件，当前暂不可行。

方案 C：上位机用实测电流恢复 setpoint（过渡方案）

串口重连后，等待 STM32 正常上报的实测电流（通过已有 parseMeasuredFromLine() 解析）。如果上位机本地 setpoint 仍为初始值，则用实测值反推 setpoint：

通道	恢复规则
L1	measured mA 四舍五入到 1mA
L2	measured mA 四舍五入到 1mA
L3	measured mA 按 100mA 对齐，且不低于 800mA

安全限制：

1. 仅在重连/启动后的恢复窗口内自动同步一次，运行中不允许用实测值持续覆盖 setpoint（否则会扰乱手动调节和缓升流程）。
2. 实测值明显异常时（L3 < 800mA、超过上限、或偏差过大），不自动恢复，仅显示实测值并提示用户确认。

限制：依赖 STM32 主动上报电流。根据 11.2 节分析，如果下位机温度无变化且无命令注入，重连后可能收不到电流信息，此时该方案也无效。

11.4 当前策略

由于 STM32 v1.3 已支持 A/B/C/D 状态查询命令，当前推荐的分级策略为：

层级	方案	状态
短期 (已实现)	每次真实串口连接或自动重连成功后，延迟 500 ms 发送一次 D 查询；若被限速挡住，每 200 ms 最多重试 5 次；根据 [LaserX] READY/SET_I/FB_I 回包同步 ready、setpoint 和实测值；同步完成前禁止用户操作	已实现
长期 (推荐)	在现有 D 查询基础上继续补充命令序号、ACK/NAK、CRC、超时重试和周期状态心跳	依赖 STM32 固件继续完善

实现路径：

- `openSerial()` 在真实串口打开成功后触发 `scheduleInitialStatusQuery()`。
- `scheduleInitialStatusQuery()` 延迟 500 ms 后调用 `runInitialStatusQuery()`。
- `runInitialStatusQuery()` 通过 `queryAllLaserStatus()` 发送 D 查询；若遇到发送限速，则每 200 ms 重试，最多 5 次。
- `parseSetpointFromLine()` 收到三路 [LaserX] ... SET_I=...A 后，才将 `lowerDeviceStateSynchronized` 置为 true 并开放控制权限。
- `canAdjustLaser()` 在真实串口模式下会先检查 `lowerDeviceStateSynchronized`；同步完成前普通页、开发者页、TRY 和 ramp 都不能发控制命令。
- 自动重连成功也会重新进入 `openSerial()`，因此同样触发初始状态同步。

11.5 安全规则

无论采用哪种恢复方案，都必须遵守以下规则：

1. **恢复前禁止操作：**在 setpoint 未与下位机实际状态同步之前，所有激光器控制命令入口保持锁定；同步完成后再按顺序解锁开放操作。
2. **优先安全方向：**L3 降低是最安全的方向——不破坏联锁关机顺序，且降功率不会造成过流风险。
3. **恢复动作需人工确认：**自动探测/恢复仅在非破坏性方案（方案 B）中允许。任何涉及控制命令的恢复动作（如发送 '9'）必须由操作员明确确认后执行。
4. **恢复失败保持未知：**如果恢复命令超时、收到错误、或回显解析失败，界面继续保持"状态未知"标记，并提示用户检查硬件连接和串口通信。
5. **实测值异常不盲从：**如果收到的实测电流明显超出合法范围（如 $L3 < 800\text{mA}$ 、 $L3 > L3\text{MaxMa}$ ），不应用其恢复 setpoint，而是标记异常并等待用户判断。

附录：更新记录

2026-06-10

普通页面 L3 百分比回显稳定性修复

- 新增普通页面 L3 百分比档位匹配逻辑： `operatorPowerPercentToMa()` 将百分比映射为按硬件步长对齐的中心电流， `operatorPowerMaToPercent()` 扫描档位表并选择距离当前 setpoint 最近的百分比。
- 新增约 ± 30 mA 的档位容差。 `operatorPowerMaMatchesPercent()` 用于判断当前 L3 setpoint 是否已经属于用户请求的百分比范围。
- `requestOperatorPowerPercent()` 在当前电流已经落入目标百分比容差范围时直接视为完成，不再为了不存在的精确电流值重复调节。
- L3 ramp 解析 `SET_I` 回包时，只要当前 setpoint 距离目标不超过 30 mA 即判定到达，避免硬件每步存在约 ± 30 mA 浮动时持续追逐精确目标。
- 普通页面新增最近请求百分比记忆：L3 调整期间以及进入目标容差范围后优先保持用户选择的百分比，不再被实时 setpoint 回显改成相邻百分比。

生产模式 setpoint 回包驱动

- `processRampStep()` 中本地 setpoint 推进和 ramp 完成判断改为 `#ifdef DEBUG_MODE` 限定：生产模式下 ramp 不再本地计数步进，完全等待 `parseSetpointFromLine()` 收到 STM32 `SET_I` 回包后推进 setpoint 并判断是否到达目标。
- `adjustLaser()` 单步命令同理：生产模式下不再本地更新 `currentLaserMa()`，setpoint 由 STM32 回包中的 `SET_I` 字段校正，避免 `serialPort->write()` 成功但 STM32 实际未执行时上位机 setpoint 虚假推进。
- `parseSetpointFromLine()` 在非 DEBUG 模式下直接承担 ramp 进度通知和完成判断：收到 `SET_I` 后 emit `operationProgress`，当 setpoint 到达目标（L3 含 ± 30 mA 容差）时调用 `finishRamp()`。
- `commandAckPending[3]` 在非 DEBUG 模式下阻止 `processRampStep()` 在前一命令尚未收到 `SET_I` 回包前继续发送下一步，形成发送-回包-再发送的闭环节奏。

开发者页面状态显示统一

- 开发者页面 `developerTemperatureBypassWarningLabel` 统一为 `developerStatusDisplayLabel`，与普通页面一样显示四种状态：未连接（灰）、同步中（黄）、旁路警告（橙）、已同步（绿）。
- 串口按钮 `openBt` 增加"同步中"中间态（橙色背景），tooltip 提示"等待下位机 D 查询回包完成，期间禁止操作激光器"； `lowerDeviceStatesSyncChanged` 信号触发后自动刷新按钮文字和样式。
- 开发者页面三路电流 SpinBox 在 `lowerDeviceStatesSynchronized` 为 false 时保持禁用，同步完成后才开放手动输入和步进。
- TRY 扫描入口 `on_tryButton_clicked()` 增加同步完成检查：未同步时弹窗提示"请等待下位机状态查询成功后再开始 TRY 扫描"，防止在 setpoint 尚未与硬件对齐时启动自动扫描流程。

默认关闭 DEBUG_MODE

- `lasercontroller.h` 中 `#define DEBUG_MODE` 改为注释状态，默认以真实串口协议编译。调试 UI 流程时需手动取消注释。生产环境必须关闭此宏，否则不会真正通过串口控制硬件。

LaserChart 动态绘制完善

- Y 轴 "mA" 标签、阈值线标签 ("阈值 XXX")、X 轴时间刻度标签、右上角设定值/实测值读数统一改为基于 `QFontMetrics` 动态计算绘制区域宽高和位置。
- 阈值标签 Y 坐标做 `qBound(marginTop, ty - textHeight, marginTop + plotH - textHeight)` 约束，避免阈值线靠近绘图区顶部或底部时标签溢出裁剪。

- 时间刻度标签居中对齐逻辑改用 `labelwidth / 2` 偏移，替换原先写死的 `-25` 偏移量。
- 右上角读数行高由写死 `14px` 改为 `valMetrics.height() + 2`，对齐方式补充 `Qt::AlignVCenter` 垂直居中。

构建系统修复

- `serialhelper.pro` 中 `QMAKE_POST_LINK` 从多行 `win32 { } else { }` 大括号块改为单行 `win32: / !win32:` 条件赋值语法，解决 qmake 解析器在处理嵌套 `$$quote()` 和 `$$shell_path()` 时的 brace 匹配 bug。
- `HEADERS` 补充缺失的 `ui_operatorform.h`。

2026-06-05

README 与当前源码结构同步

- 补充程序入口：`main.cpp` 初始化高 DPI、UTF-8 locale 和全局字体后直接启动普通操作员页面。
- 补充普通页面和开发者页面关系：开发者入口经过密码验证，`widget` 与 `operatorForm` 复用同一个 `LaserController`。
- 补充 `LaserChart` 的设定值/实测值双曲线和 `PillSpinBox` 的百分比功率输入行为。
- 按 `serialPortReadyRead()` 当前实现收紧温度/就绪解析说明：异步温度事件仅覆盖 L1/L2，L3 主要依赖 `[Laser3] READY=YES/NO` 状态回包。
- 修正全局限速说明：真实串口控制帧和状态查询帧共享 `globalSendTimer`，不是三路完全独立并发发送。

STM32 v1.3 文本帧协议适配

- 上位机真实串口发送从裸单字节改为 `<CMD=X,CHK=YY>` 文本帧，`CHK` 为单字符 `CMD` 的 ASCII 低 8 位十六进制值。
- 控制命令仍沿用 `0~9`，新增/启用查询命令 `A/B/C/D`，其中 `D` 用于查询全部激光器状态。
- 每次真实串口连接/自动重连成功后延迟 500 ms 发送一次 `D` 查询；若被限速挡住会每 200 ms 最多重试 5 次；空闲状态下周期性查询，保持 ready、setpoint 和实测值刷新，避免上位机默认初始状态覆盖下位机实际输出状态。
- 新增 `lowerDeviceStateSynchronized` 和三路状态掩码；连接后先显示“状态同步中”，只有收到 L1/L2/L3 三路状态行并更新本地状态后才显示“状态已同步”并开放控制权限。
- 普通页面连接栏只显示连接端口；下位机查询状态移动到温度旁路警告预留区域。同步完成前该区域优先显示查询进度，同步完成后再决定是否显示温度旁路警告。
- `sendLaserCommand()` 和 `sendStatusQuery()` 均检查 `bytesWritten == data.size()`，避免部分写入被误判为成功。
- `sendStatusQuery()` / `queryAllLaserStatus()` / `queryLaserStatus()` 改为返回 `SendResult`，便于区分 `Sent`、`RateLimited` 和 `Error`。

状态回包校正与 pending 机制

- 上位机解析 STM32 v1.3 状态行 `[LaserX] READY=... SET_I=...A FB_I=...A`。
- `READY=YES/NO` 更新 `rawReady` 并重新计算依赖联锁；`FB_I` 更新实测曲线。
- `SET_I` 用于校正上位机 `currentLaserMa()`，不再仅凭 `serialPort->write()` 成功推进 setpoint。

- 新增 `commandAckPending[3]`，真实串口模式下同一通道会等待对应状态回包后再发送下一步 ramp 命令。
- 收到 `ERROR` 时清除 pending；若 ramp 正在执行，则中止 ramp。
- `DEBUG_MODE` 保留用于界面调试；调试模式仍走本地模拟推进，不影响 UI 调试流程。

界面 DPI / 尺寸保守修复

- `main.cpp` 增加全局 `Microsoft YaHei` 字体，稳定中文字体 fallback。
- 开发者页为 3 个电流 SpinBox、串口按钮和 TRY 按钮补充最小宽度与横向伸缩策略。
- 普通操作页为 L1/L2 主按钮、串口下拉框、串口连接按钮和功率 SpinBox 补充最小宽度与伸缩策略。
- 温度旁路警告标签保留自动换行，降低小窗口或高 DPI 下文字裁剪风险。
- `LaserChart` 自绘文字区域改为基于 `QFontMetrics` 计算，减少 mA、阈值、时间刻度和右上角读数的裁剪/重叠。

2026-05-25

重启恢复策略分析

- 新增 [11. 重启恢复策略](#)，分析上位机意外关闭后状态丢失问题及三种恢复方案。
- 方案 A（降低 L3 探测状态）已否决：控制命令不应作为查询手段，存在副作用。
- 方案 B（下位机 '?' 查询命令）定为长期正式方案，需 STM32 固件配合。
- 方案 C（实测电流同步 setpoint）定为过渡方案，但受限于 STM32 非周期性上报特性。
- 当前短期策略：重连后进入"状态未知"模式，禁止升高；提供"安全恢复 L3"按钮供用户确认后执行。
- 明确恢复安全规则：恢复前禁止升高、优先安全方向、人工确认、失败保持未知、异常值不盲从。
- README 新增目录，所有章节可通过锚点跳转。

代码修复

- `sendLaserCommand()` 改为三态 `SendResult` 枚举，`processRampStep()` 限速时跳过本次 tick 等待重试。
- 删除未使用变量 `numSteps` 和 `continueOperatorSoftOn()` 重复注释。

2026-05-23

L2 启动曲线简化（三段式 → 单段缓升）

- L2 从 `0→850→200→90mA` 三段式改为 `0→460mA` 单段缓升（步长 10mA，时长 30s）。
- 移除 `TryPhaseL2Start1/2/3` 三个 TRY 状态，`TryPhaseL2` 直接执行单段缓升。
- `requestOperatorSwitch(2, true)` 改为调用 `setLaserTarget()` 而非 `startOperatorSoftOn()`。
- L2 参数弹窗从 10 个控件精简为 3 个（目标电流、缓升时长、步长）。

L2 参数统一化

- 普通页面 L2 开启和 TRY L2 扫描共用同一套 `[StartupL2]` 参数（`FinalMa`、`RiseDurationMs`、`StepMa`）。
- L2 参数弹窗不再暴露独立的 TRY 目标/步长/时间字段，保存时自动同步。

- TRY 扫描弹窗中 L2 参数改为只读显示，标注"来自 L2 参数"。
- `loadConfig()` 启动时强制 `tryL2TargetMa = startupL2FinalMa`，旧 ini 的 `DeveloperTry/L2*` 降级为兼容镜像。

全局发送限速 15Hz (67ms)

- `defaultRampIntervalMs` 150→67ms, `minRampIntervalMs` 1→67ms, `minManualSendIntervalMs` 120→67ms。
- `widget.cpp` 新增 `kTryMinIntervalMs = 67`，`tryIntervalForSegment()` 下限从 1ms 提高到 67ms。
- 三层限速统一锁在 15Hz，消除 ramp 间隔小于 `sendLaserCommand` 硬地板导致的发送失败风险。

普通页面功率步进

- 新增 `operatorPowerPercentStep()`，确保 +/- 一次跨过至少一个 L3 硬件电流档位 (100mA)，避免按键表现为无效。

按钮闪烁修复

- 新增 `visualRefreshTimer` (零毫秒单次定时器)，合并同一事件循环内的多次 `updateAllLaserVisuals()` 调用。
- TRY 扫描期间通过 `anyBusy || tryRunning` 锁定手动操作按钮，避免 ramp 段间隙短暂解锁。

ramp 限速重试机制

- `sendLaserCommand()` 返回值从 `bool` 改为三态枚举 `SendResult { Sent, RateLimited, Error }`。
- `processRampStep()` 在 `RateLimited` 时跳过本次 tick (等待 `rampTimer` 下次触发重试)，不再因发送间隔过短而直接中止 ramp。
- `adjustLaser()` (单次手动 +/-) 将 `RateLimited` 和 `Error` 统一视为失败，保持单次命令语义。

代码清理

- 删除 `widget.cpp` 中未使用的 `numSteps` 变量。
- 删除 `continueOperatorSoftOn()` 中两处重复注释。